

Transcript

Module 8 Week 10: Working with React - Part 2

Video 1: Working with React Events

Welcome to the lecture on "Working with React Events". Here are the learning objectives. By the end of this lecture, you should understand what HTML events are and how to handle events both in a general webpage context and within React.

Let's begin with a quick overview of HTML events. HTML events are actions that occurs in the browser, such as a mouse click, mouse movement, or form submission. These event can trigger specific behaviours on the webpage. For instance, when we click a button, the `onClick` event is fired. Other common events include `onmouseover` for mouse movement and `onkeydown` for keyboard interaction. To illustrate how to handle HTML events consider this example. Here the `onClick` attribute specifies an action which should occur when the button is clicked. In this case, it triggers a JavaScript function: the `alert` function, which displays a message box on screen showing button is clicked.

Handling events in React is similar but comes with a few slight differences. React events are written with camelCase so instead of `onClick`, you will have to write it with `onClick` with a capital C. Additionally, instead of passing a string we pass a function reference as the event handler. For example, in this code `onClick` is the event and `sayHello` is the function reference that will be called when the button is clicked. It's crucial to note that we pass a function reference without parentheses to avoid invoking it immediately when the component renders.

We can define nested functions within our components to generate JSX and trigger other logics. Here is a simple example. In this component the `sayHello` function is defined within the `SayHelloComponent`. The `sayHello` function is then triggered when the button is clicked. When working with textboxes in React we can handle changes using the `onChange` event. We will need to specify an argument often named `event`, to assess the event object. This object can then be used to retrieve the value that is currently being typed into the textbox.

Here is how you can do it? To assess what is being typed into the textbox we can use the `target.value` attribute of the event object. Here is an example to illustrate this idea. Every time the user types into the textbox the event handler `printContent` function will be called, which prints the current value of the textbox to the console.

For handling form submission React provides the `onSubmit` event. You can use it to perform actions when a function is submitted such as making network requests or processing form data. Consider this example. In this component the `handleSubmit` function is called when the form is submitted. The `event.preventDefault()` method prevents the default browser behaviour which in this case is a full page reload. It is used to prevent the default action of an event. In the context of form submission it stops the browser from actually submitting the form which leads to a reloading of the page. Instead, we hijack this user action and perform the action programmatically.

To summarise, we have covered the essential of handling events in both HTML and React. You should now understand how HTML event are and how to handle them in a webpage as well as how to handle events effectively in React.

This foundational knowledge will be crucial as you continue to build more interactive and dynamic React applications.

Video 2: Working with React Hooks and States

Welcome to the lecture on working with React Hooks and states. Here are the learning objective. By the end of this lecture you should have a good understanding of the differences between props and states, what React Hooks are and how to define state variables using Hooks.

In React, two types of data control a component: props and states. Props are often used for the initial rendering of a component but the UI often changes over time due to user interaction and other events. While props are fixed and supplied by the parent component the state is mutable and can change over time.

One of the most powerful features of React is its auto-rendering capability. When the state of a component changes React automatically re-renders the component to reflect these changes. For instance, imagine that we have a state variable called `textColor`. If this state changes the colour of the text in the component will automatically update. Before the introduction of React Hooks in version 16.8 managing states in React requires using ES6 class component.

Function components are known as stateless functions because they couldn't hold states. Here is an example of how state was initialised in class component. To change the state in class component we use the `this.setState()` method. This method allows React to check if there is a need to re-render any part of the UI. We often start with a stateless function and rewrite it as a class component when we need to manage state. However, using classes in JavaScript can lead to unexpected behaviour especially for new programmers due to the complexity of the `this` keyword.

Since React version 16.8 we can use Hooks to manage states without writing a class. This means function components can now hold state. If we need to use state we don't have to rewrite the entire component. We can simply add the state logic into our function component resulting in shorter and cleaner codes. Hooks are functions that let you Hook into the React state and lifecycle features. They allow us to use React without classes. The React team has been advocating this approach since its introduction. Although class components are still available future support might be limited.

Many code examples still use class components but Hooks are becoming increasingly popular. Hooks typically follows a naming convention that starts with the use prefix followed by the Hook description. While they look like normal JavaScript function their behaviour are special, hence the name Hooks. React provides server built-in Hooks but developers can also create their custom Hooks. The most common Hooks are `useState()`, `useEffect()`, and `useRef()`. To use Hooks like `useState()` we need to first import the `useState()` function from the React library. This is different from importing a React component from a local file. Here we are importing from a Node module. The difference between the two importing style is whether there is a `.slash` prefix before the module name.

Let's take a closer look at the `useState()` Hook. First we import `useState()` from the React library. Next we initialise a state Hook and specify an initial value such as `useState(0)`. This Hook returns an array with two items. Often we perform array destructuring to extract these two values into two separate identifier. The first value is the current state and the second value is a function that is used to modify the state value. Here is an example of initialising a state Hook and using the array destructuring construct.

In this counter example each time the state changes the function is triggered again and the UI is potentially rerendered. We initialise a count state variable with an initial value of 0. We then define an event handler that increment the count each time a button is clicked, and update the count state variable by calling the `setCount()` function and supplying a value to be assigned to the count state variable. React will then retrigger this function with the updated count state variable which will lead to rerendering of this component. This is how the updated count value is reflected in the UI.

It's important to only modify state values using events or condition. Writing codes improperly can lead to an infinite loop causing the programme to crash. Here is an example of what not to do. In here, we modify the counter state variable within the body of the function component without any condition or event. Since the changes in state variables will lead to the function getting retriggered this will lead to infinite rerendering. When naming state variables follow normal variable naming conventions. The setter function is typically named with a `set` prefix followed by a state variable. You should treat the state variable as like any other variable in your code.

Remember to assign a function reference to your event handler instead of doing a function call. If a function call is used that part of the code will be triggered during rendering, even without the user clicking on the button. Here is an example that demonstrates the use of `useState()` Hook to manage the state of a textbox field. When the input value changes the state is updated and the UI reflects the new state. This example also shows conditional rendering when an element is only rendered if a specific condition is met. Conditional rendering such as the following is quite common. The JSX will only be rendered if the specific condition is true. This helps to create dynamic and interactive UI based on the application state.

To summarise, we have covered the differences between props and state, what React Hooks are and how to define state variables using Hooks.

Video 3: Controlled vs. Uncontrolled Components

Welcome to a lecture on "Controlled vs. Uncontrolled Components". Here are the learning objective. By the end of this lecture, you should understand the two types of components in React and be able to differentiate between Controlled and Uncontrolled components.

In React, components are rendered according to the state or prop. The two main kinds of components when dealing with states are controlled components and uncontrolled components. Control components render the user interface based on the state values. At any given time, the state values dictate the UI. For example, consider a textbox whose value is bounded to a state variable. When you type into the textbox, it will update the state value for each `onChange` event. And since the textbox is bounded to the state variable, this will effectively update the value of the textbox.

Controlled components are generally recommended because the state serves as the single source of truth. This makes it easier to test your components since you can always verify the UI by checking the state values. When the state values are correct, you can be confident that the UI will be correct. Uncontrolled components, on the other hand, update the state at specific events point, such as the result of an `onClick` event. In this case, the DOM acts as the source of truth. Which is not ideal. Consequently, for unit testing, you can't rely solely on checking the changes in the state values. You will also have to check the DOM values, whether they are rendered properly.

Let's take a look at an example of an uncontrolled component. In this scenario, the DOM itself maintain the state until a specific event triggers a state update.

Specifically, when a user types a name into a text box, what the user type in is reflected in the text box because that is the default behaviour of text boxes. However, each keystroke is not captured into a React state variable. Instead, the state variable is only updated when the user press the update name button. The issue with this approach is that the UI is not just defined by the state variable but also defined by the contents of the DOM. For example, the page can show Hello John, but the textbook might contain another name that is not John, since the state variable is not modified yet.

To summarise, we have covered two types of components in React, controlled components and uncontrolled components. We have discussed the key differences between them. Controlled components use the state as the source of truth. And uncontrolled components relies on the DOM. Understanding these key differences will help you make informed decision when building your React application. As far as possible, it's always preferred to use the controlled component approach.

Video 4: Asynchronous JavaScript and XML (AJAX)

Welcome to the lecture on "Asynchronous JavaScript and XML AJAX". Here are the learning objective. By the end of this lecture, you should be able to understand what JavaScript Object Notation, or JSON is, what Asynchronous JavaScript and XML or also known as AJAX is, and finally you should understand how to work with the `fetch()` function, which is a modern and efficient way to perform AJAX requests.

Let's start by mentioning that the web is adopting a server-client architecture. In a typical web application, the client, which is the web browser, makes a request to the server for data. This data can be anything from user information to the contents of a blog post. The server processes this request and sends back the appropriate responses. Understanding this interaction is crucial for working with AJAX as AJAX essentially enhances this communication process by making it asynchronous.

Before we discuss further about AJAX, let's talk about the Java Object Notation or JSON for short. JSON is a lightweight data exchange format used for transferring data between a server and a client. A data exchange format is a standardised structure for representing data. Instead of capturing just a single value, many of the data exchange format is able to encode structural information. For example, we could capture the addresses of a person record and each address record could further consist of multiple components. Despite its name, JSON is language independent and can be used to store data in other programming languages. Since data in JSON is always in text format, this makes it easy to transmit across the network.

Even though the syntax of JSON resembles JavaScript Object literal and array literals, there are some key differences. For example, here is a JSON string. One of the key differences is the property of the object needs to be enclosed using the double quotation marks. Now to convert this string into a JavaScript object, we use the `JSON.parse` function. Conversely, if we have a JavaScript object that we want to convert to JSON string, we can use the `JSON.stringify()` function.

Now let's dive into AJAX. AJAX is a technology that allows web pages to update asynchronously by communicating with the server programmatically. Unlike traditional communication which involves doing a form submit and refreshing the entire page, AJAX allows for parts of the web page to be updated without reloading the entire page.

AJAX requests are made using JavaScript, and the server will return the requested content which can be in various formats such as text, HTML, XML, or JSON among which JSON is perhaps the most commonly used for this purpose.

In modern web development, client often interact with the server through web services such as the RESTful APIs. These services provide endpoints that the client can send requests to and receive responses from using AJAX. Here is an example of a JSON response from a web service. This example comes from the following URL. The data includes information about the person such as name, username, email, address, and company details. This structured format makes it easy to pass and use the data in our application.

Traditionally, making AJAX requests involved using XMLHttpRequest object which could be quite cumbersome. Modern browser now provides this `fetch()` function, which simplifies the process significantly. The `fetch()` function returns a promise that resolves to an object representing the response of a request. Here is an example of how to use a `fetch()` function. The `fetch()` function makes use of Promises, which are an important concept in modern JavaScript development.

If you are not familiar with promises, I highly encourage you to read more about them as they are widely used in asynchronous programming. To wrap up, we have covered the fundamental of JSON, the concept of AJAX, and how to use the `fetch()` function to perform AJAX request.

Video 5: Working with useEffect Hooks

Welcome to the lecture on working with `useEffects` Hook. Here are the learning objective. By the end of this lecture you should understand how to work with the `useEffect` Hook. Effect Hooks or `useEffects` are used to perform side effects in your component. The most common side effect you encounter is fetching data. Why do we need a special hook to handle fetch requests? If you simply put the fetch logic inside the component function it will trigger every time the function is called which happens with every state change due to the re-rendering feature of the React. This can lead to unwanted communication with the server and performance issue.

Let's first look at a bad example of data fetching. Here the fetch logic is placed directly inside the component function body without using the effect hook. Consequently, each time when the function is called due to a rendering or re-rendering this fetch function will be executed. We can see how inefficient this is by examining the following code sandbox link. This is something we want to avoid. To ensure that our fetch logic runs only when necessary we use a effect hook.

Like `useState`, we need to first import the `useEffects` from the React library. This will allow us to call the `useEffects` function within the function body. Then to ensure that our fetch logic runs only when necessary we can then call the `useEffects` function and provides two argument. The first argument is a function definition with the logic to be executed. The second argument is a dependency array which will decide when the function as defined in the first argument will be triggered. In this code example, when the value of `size` changes the function defined in `useEffects` will be triggered. It will also be triggered for the initial value of `size`. Try pausing this video and visit the following code sandbox link to investigate this behaviour.

Another important use case for `useEffects` is running some codes only once. To do this, we provide an empty dependency array. This tells React to run the effects only once for the initial load. This is perfect for the initial loading of data. Here is a code sandbox example demonstrating how to fetch data on initial load. If you go to the link, you will realise that the empty array prevents the effects hook from running on subsequent render making it perfect for one-time operations.

When using `useEffects` for data fetching we often update a state variable with the fetched data. It is important to always specify a dependency array to ensure that this fetching procedure is based on some condition. If not, it might lead to infinite re-rendering. It's possible to use multiple effects hooks within a single component. Each `useEffects` can handle different side effects making your codes modular and easier to read. For instance, one `useEffects` might fetch the initial data while another fetches data based on a query condition.

To summarise, we have covered how to work with `useEffect` hooks in React. We have explored the importance of managing side effects, how to fetch data efficiently and how to avoid common pitfall. By mastering `useEffects` you will be able to build more robust and efficient React applications.

Video 6: Working with `useRef` and Rules of Hooks

Welcome to a lecture on working with `UseRef` and rules of hooks. Here are the learning objectives. By the end of this lecture, you should have a good understanding of how to effectively use the `useRef` hook. We will also discuss the rules of working with React hooks. We have encountered `UseRef` earlier in our discussion about uncontrolled components.

Now, let's explore it further. The `UseRef` hook is particularly useful when we need to reference specific HTML elements such as for the purpose of doing animation. In this example, we create a reference to a text box by initialising `textboxRef` and binding it to the `ref` attribute of the text box. Similar to the `useState` and `useEffects`, we will need to import the `useRef` function from the React Library.

Subsequently, to obtain the reference to a HTML element, we use the `.current` property of the `ref` variable. Here is an example. Let's look at this [code sandbox link](#) to see how `useRef` works. Pause this video at any point in time to investigate its behaviour. In this example, clicking the toggle button toggles the `show` state value which defines the visibility of the text box. If suppose the previous value of `show` is `false` when this toggle action happens, the `show` will be changed to `true`. We then use the `textboxRef` to gain reference to the text box and trigger the `focus()` method which will focus the text box so that the user does not have to manually click on the text box to focus the cursor on the text box. This allows for a better user experience.

There is just a few specific details to take note. First, note that when the setter methods for state variables are called, the new values of the state variables only get reflected at the end of that method. In this case, `setShow` is called in the `onClick` handler of the button, while at line 12, `setShow` toggles the value of `show`. This does not mean that line 13 will see the new value of `show` immediately. Instead is only when the body of the `onClick` is all executed that the React engine will update the state values. This is the reason why line 13 might look rather strange since the `if` condition does the focus logic when the `show` value is `false`.

Consequently, as the updating of state value, happen shortly after this function is triggered, this will result in a re-rendering of this component. So this function will be called again. In which case, the conditional rendering of line 9 will cause the textbox to be displayed. So in the body of the `if` condition, that is from line 14 to 16, we will delay the focusing of the textbox by a split second.

By delaying by 100 milliseconds, we assume that the re-rendering has already taken place, and the DOM already contains the textbox element so the focus will work. If you were to call the `focus` function before the textbox element is presented in the DOM, nothing will happen. When you assess the [code sandbox link](#), I would encourage you to try editing the code over your end to investigate the different behaviours that I have just mentioned.

Besides referencing HTML elements, `useRef` can also be used to create mutable variables. Unlike normal variables which are reset during re-renders, a variable created using `useRef` can persist its value across renders. This is particularly useful for storing data that needs to be updated without triggering a re-render. As we have seen in the earlier examples, hooks in React are not just regular functions or variables. They have specific behaviours that work differently from normal JavaScript functions.

There are rules that govern their usage. It's crucial to follow these rules to ensure that your components work correctly. The rules of hooks are straightforward but essential. First, you should only call hooks at the top level of your React function. Do not call them inside loops, conditions, or nested functions. Second, only call hooks from React functions. This means they should only be used inside functional components, not any regular JavaScript functions defined outside of the functional components. Here is an example of incorrect usage where a hook is used in a condition. For more details, refer to the official Rules of Hook website.

To summarise, we have covered how to work with `UseRef` Hook and the critical rules of working with React hooks.

Video 7: React Router

Welcome to the lecture on "React Router". Here are the learning objective. By the end of this lecture, you should understand how to use React Router to define different pages in single page application.

Let's start with the concept of Single-Page Application, or SPA. Frontend frameworks like Angular, React, and VueJs allow us to create SPAs, where there is technically only one page. However, often we want to create the effect of multiple pages, similar to traditional websites. To provide the illusion of multiple pages, the solution lies in dynamically switching the UI components based on the user navigation path. The main benefit of SPAs include no full page reloads, leading to a smoother user experience and reduced bandwidth consumption.

Frontend routing follows a similar idea to backend routing but is implemented on the client side. The client scripts will determine which component to render based on the URL path. The 2 common routing mechanism in React are `HashHistory` and `BrowserHistory`. When working with `HashHistory`, URLs look like the following. Specifically, the URL contains a hash symbol followed by the path. The main page will then display the relevant components meant for that path. On the other hand, when using `BrowserHistory`, URLs resemble those used in backend routing. Note that even though the URL contains the full path, the server will access the main page and the script in the main page will then interpret the path from the URL and display the relevant component.

To achieve this routing capability, we install a third-party library known as React Router. This is a command to install the library into your React app. To set up React Router, you need to first wrap your main components within either the `HashRouter` or `BrowserRouter` component. Specifically, within `index.js`, we will wrap our `App` component within one of these Router components.

To understand how React Router works, we will be following this [CodeSandBox URL](#). Try accessing this URL on your machine and follow along with this discussion. As shown in the previous screen, the app component consists of a sidebar and a main component. By clicking on the link on the left sidebar, the right panel which consists of the main component will reflect the page that the user wants to visit.

Within the Main component, we then use the Route element to define which component to render for a specific path. For example, the profile path will be replaced by the profile component. The component is defined in the element attribute of the Route component. Since we only want the main component to show one component depending on the path, we can use the Routes component to help select the first matching path. The Routes element works like a Switch statement in programming and the Route element corresponds to the case statement within the switch construct.

Suppose we click on the home link on the left sidebar which corresponds to the slash path. That is the one without any path name, it should replace the Routes component with the Home component, giving the user the illusion of the home page. Now suppose we click on the Programming link, the path would be different and the Routes element would be replaced by the element defined for the programming path. To navigate to different paths, we use the Link component that is defined in the React Router package. Note that unlike a normal hyperlink, this component prevents the default behaviour of a hyperlink which does a full page reload. Hence the navigation will appear smoother this way.

Instead of using the Link component, it is also possible to use the NavLink component if we want to do styling to our links. Specifically, the NavLink component will add an active class to the hyperlink that matches the current URL. In the CodeSandBox example, we apply a bold style to the active class which provides a visual cue of the current page.

So far, the paths that we are navigating to are fixed. Sometimes we might want to render a component differently depending on its routing parameters. For example, we could define a modulecode route parameter which specifies the module code that we are interested to see. Under the routes component, we define the router parameters pattern and specify the Module components for such path. Within the module component, we can then access the route parameters using the useParams hook. To better understand how this works, click on the Programming link on the sidebar and then click on the links on the programming page. Observe that the URL follows the patterns that we have discussed earlier.

So far we have been accessing different pages by clicking on the links. There are situations where we might want to programmatically navigate to a URL. For example we might want to navigate to a confirmation page if an action is successful. We can use the useNavigate hook to achieve this. By calling useNavigate and specifying the path will allow us to programmatically access a path or go back in the browser history.

To summarise, we have covered how to use React Router to define multiple pages within a single page application. We have explored setting up routers, defining paths using links and navigation components, handling route parameters, and accessing routes programmatically.

Video 8: Create React App

Welcome to the lecture on "Create React App". Here are the learning objective.

By the end of this lecture, you should understand how to use Create React App to create React applications. Create React App is a tool that is commonly used to create React applications. You will also understand the purpose of using Create React App. When we talk about production-ready applications, we are referring to applications that are optimised for deployment.

Let's recap on what transpiling is about. Transpiling allows us to transform our codes into another form. For example, previously we transpiled JSX codes into JavaScript. Another situation where we

were to do transpiling is to convert codes which are using a newer syntax into a form that better support older browsers. For example, some ES6 construct cannot be executed on older browsers. We have seen how to perform transpiling using Babel during runtime but this is very inefficient. Ideally, we want to transpile it in advance during deployment, so that our application can run as normal JavaScript in the browser. One way to do this is by using the Babel Command line interface or Babel CLI.

Let's take a look at how you can use the Babel CLI. The first step is to install the necessary tools using the Node Package Manager or npm. The following is the command that you will execute in order to do the installation using npm. Once installed, we can write our ES6 code, such as an arrow function, and then use Babel to transpile this code into a version that does not use the more advanced ES6 construct, so that this code will still be compatible on older browsers. As we see from the code snippet, this transforms modern JavaScript syntax into a normal JavaScript function that is more widely supported even on older web browsers. Here is how you can do this on the Babel CLI to transpile JSX code into normal JavaScript. However, manually transpiling our codes every time we make a change is tedious and time consuming.

This is where Create React App comes into play. It is a tool that automates this process for us, making the development process much more efficient. With Create React App, we can enjoy features like auto-reloading when we save our code and bundling all our codes into one optimised file for deployment. Create React App provides a command-line interface that makes it easy to set up a new React application.

To create a new app, we can use the npx command, which uses the latest version of Create React App for the setup process. With just a few commands, we can set up our development environment and start building our React app immediately. Once our app is set up, we can start the development server. This server works similarly to CodeSandbox, where any changes made to the code are automatically reflected in the web browser. This instant feedback loop is incredibly valuable during development. As it allows us to see the code changes at real-time. The file structure used in Create React App project is similar to what we've seen in CodeSandbox, which makes it easy to transition between the two. In fact, the React environment in CodeSandbox was built using Create React App. For local environment, we also recommend using Visual Studio Code, which is a powerful, free code editor that integrates well with Create React App, giving you a smooth and efficient workflow.

To wrap up, we have covered how to use Create React App to create React applications, and the purpose of using Create React App. Remember, Create React App is here to make your life easier, allowing you to focus on writing great codes, rather than worrying about the underlying setup.

Video 9: Module Summary

We will be summarising the "Working with React" module. Over the past series of lessons, we have covered a range of topics designed to make you proficient in building applications using React.

We started with an introduction to React, understanding its component-based architecture, benefits such as improved performance through the Virtual DOM and setup using JSX and transpiling. We also discussed how to create simple components using either ES6 classes or function components. Next, we explore React Props, understanding how to pass data between components to make them reusable and dynamic. Props are vital for making components adaptable, and we have learned how to evaluate JavaScript expressions within JSX using the curly braces. We then discuss ES6 Module System, learning how to organise React applications by separating each component into their own files, exporting, and importing these components for better code maintainability.

We also introduced CodeSandbox for a no-setup-needed development environment. To write cleaner and more efficient code, we covered Modern ES6 Constructs such as Arrow Function, Template Strings, Object Literal Shorthand Notation, Rest and Spread Operators, and Destructuring. These constructs are frequently used in React to simplify coding.

A hands-on exercise followed, where we built a profile viewer from scratch using CodeSandbox. This exercise helps reinforce concepts like component-based design, props, and basic styling using JavaScript objects. We then look at working with React events, focusing on handling user interaction like clicks, and form submissions in React. We also compared this with traditional HTML elements, noting key differences like camelCase syntax and function references.

Next, we introduced React hooks. Specifically, useState hook for managing state in function components, allowing us to move away from class components. We show how hooks enable a more concise and readable way to manage state in React. We learn about controlled and uncontrolled components. Controlled components use state as the source of truth, which makes them easier to test and more predictable, while uncontrolled components rely on the DOM, which can complicate the state management.

We then explore AJAX for making asynchronous requests, allowing parts of a web page to update without a full page reload. The fetch() function was introduced as a modern way to handle such requests, along with JSON format for data exchange between client and server. Using the useEffects hook, we learn how to handle side effects like data fetching in our component efficiently. We then delved into useRef hook for referencing DOM elements and creating mutable variables. We have also covered the rules for using hooks. These rules are critical when working with hooks.

Next, we look at React Router, which is a third party library for creating multi-page experience in a single page application. We discuss how to set up routers, define routes, using links, handling route parameters, and programmatic navigation. Finally, we explore Create React App, a tool that simplifies the creation of new React applications. It handles complex setup process and optimise our app for deployment. It also integrates features like auto-refresh and bundling for a streamlined development experience.

In summary, we have covered all the foundational aspects required to build versatile, performant React applications. From basic components creation to advanced topics like hooks, state management, and routing, you have now had the skill to develop and organise React applications efficiently. Thank you for your time and attention. Keep practicing and happy coding.

THE END